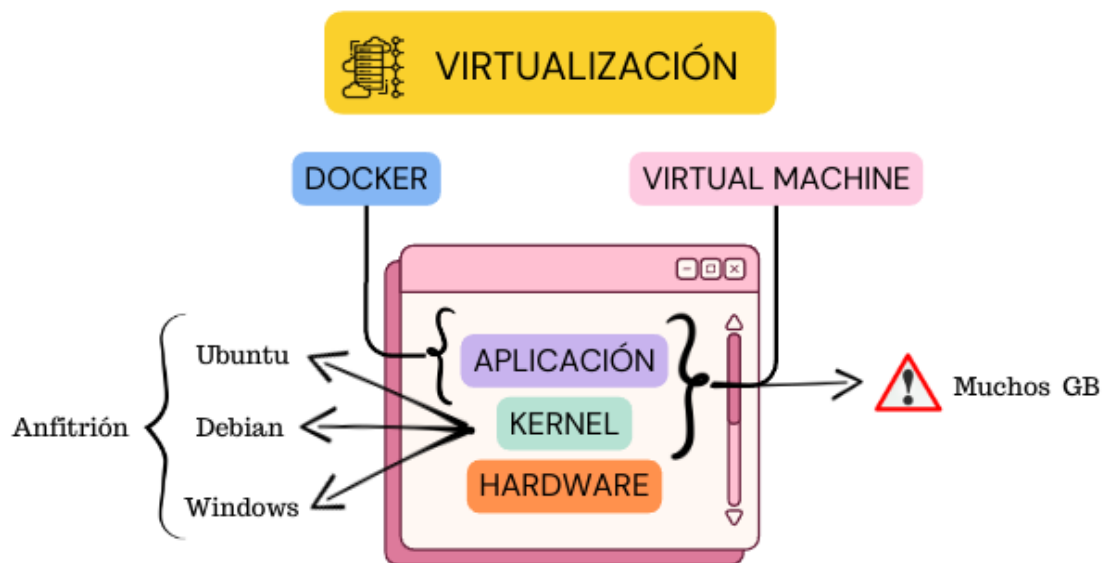


§ Apuntes de Docker §



¿Qué son los contenedores?

Es una tecnología que permite empaquetar una aplicación y todas sus dependencias necesarias para que se ejecute de manera consistente en cualquier entorno de computación. Los contenedores se utilizan ampliamente en el desarrollo y despliegue de software debido a sus ventajas en portabilidad, eficiencia y gestión de recursos.



Instalar Docker Desktop

Abrimos la página oficial de Docker Desktop en el caso de Windows y MAC simplemente se descarga se instala y listo. En el caso de utilizar linux puedes descargar el paquete correcto para tu distribución de Linux e instalarlo con el administrador de paquetes correspondiente.

¿Qué es Docker Hub?

Docker Hub es un servicio en línea proporcionado por Docker Inc. que permite a los desarrolladores almacenar y distribuir imágenes de contenedores. Es una especie de "biblioteca" de imágenes de Docker, donde puedes encontrar imágenes oficiales, imágenes comunitarias y también puedes alojar tus propias imágenes.

Ejemplo de ejecución en línea de comandos para Docker

```

1  docker images          # Listar todas las imagenes disponibles localmente
2  docker pull mysql      # Descargar la imagen de MySQL desde Docker Hub
3  clear                  # Limpiar la terminal
4  docker images          # Listar todas las imágenes disponibles localmente nuevamente
5
6  docker pull mysql:8.0   # Descargar una version especifica de MySQL
7  docker pull mysql --platform linux/x86_64 mysql # Descargar una imagen especificando plataforma
8
9  docker image rm mysql   # Eliminar la imagen de MySQL
10
11 docker create mysql     # Crear un nuevo contenedor pero no iniciarlo
12 docker container create mysql # Crear el contenedor y obtener su identificador
13 docker start <Identificador del contenedor> # Iniciar el contenedor usando el identificador
14 docker ps              # Verificar si el contenedor está corriendo
15 docker stop <Identificador del contenedor> # Detener el contenedor
16 docker ps -a           # Mostrar todos los contenedores creados
17 docker create --name MiBD mysql      # Crear un contenedor con un nombre específico
18
19         ### Exponer el contenedor a los puertos del computador ###
20 # Estructura: Comando -p(Puerto del computador:Puerto de la imagen) --name Nombre mysql #
21
22 docker create -p 27080:27080 --name MiBD mysql
23
24         ### Especificar los puertos mapeados ###
25 # En puertos tiene que especificar algo así IP:Puerto->Puerto/tcp #
26         0.0.0.0:27080->27080/tcp
27
28 ## Ejecutar el comando `create` usando `run`, que descarga, crea e inicia el contenedor ##
29
30 docker run mysql
31
32         ### Crear un contenedor con variables de entorno configuradas ###
33 # Puedes encontrar las especificaciones dentro de Docker Hub, por ejemplo: #
34
35 docker create -p 27080:27080 --name MiBD -e MYSQL_ROOT_PASSWORD=123 mysql
36
37         ### Para poder conectar contenedores entre si necesitamos una red interna de docker ###
38
39 docker network create MiRed      # Crear una red de docker
40 docker network ls               # Ver mis redes disponibles
41
42         ### Crear una imagen de docker en base a un archivo Dockerfile. ###
43
44 docker build                    # Construye la imagen
45 docker build -t miapp:1 .      # Crea la imagen y le coloca el nombre.
46
47 # Creamos la imagen con puerto, nombre, red y variables de entorno definidas.
48 docker create -p8080:8080 --name MiBD --network MiRed -e MYSQL_ROOT_PASSWORD=123 mysql
49
50 # Creamos la imagen con puerto, nombre, red y la imagen creada a partir del Dockerfile.
51 docker create -p8080:8080 --name MiBD --network MiRed miapp:1

```

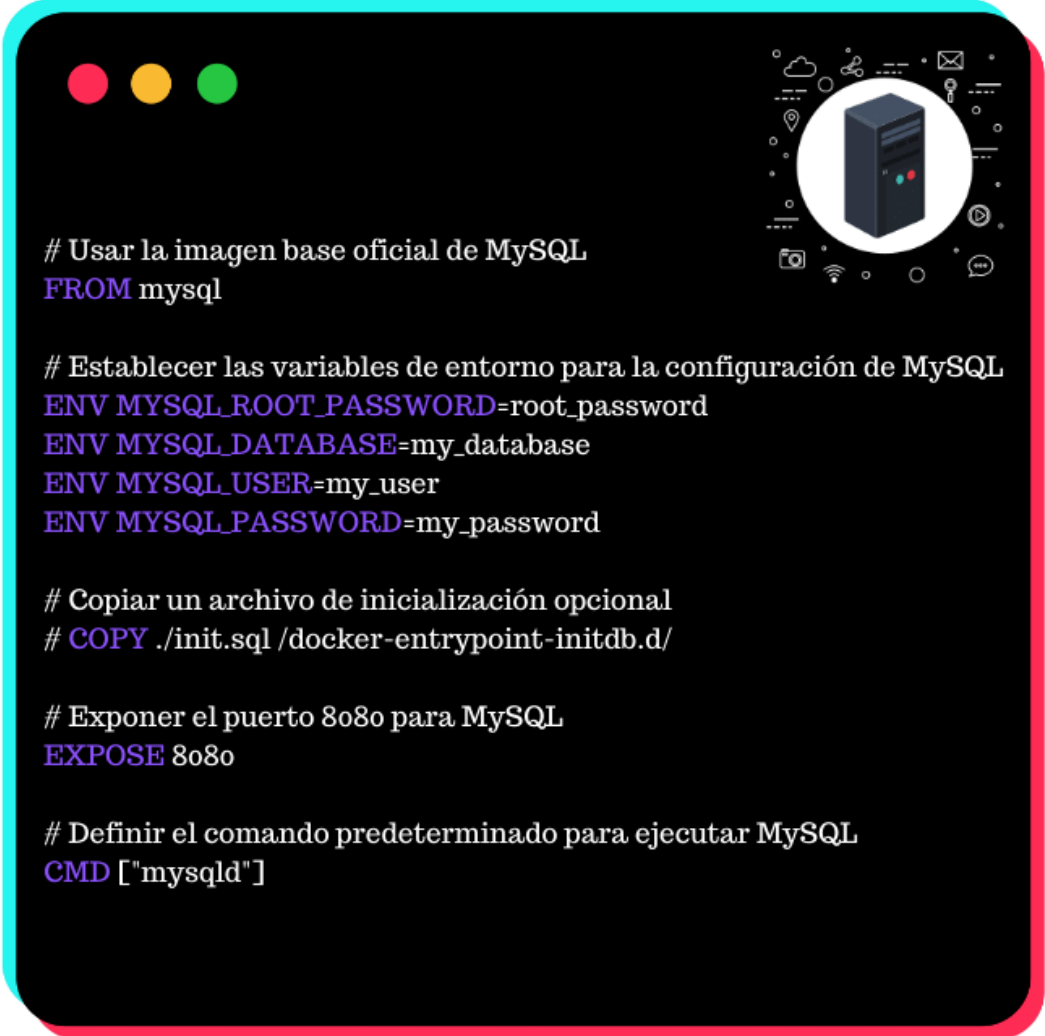
Explicación del archivo Dockerfile

El archivo de docker solo se puede llamar Dockerfile.

- FROM: Especifica la imagen base desde la cual se construirá la nueva imagen.
- MAINTAINER: Proporciona información de contacto sobre el mantenedor de la imagen. (No recomendable)
- RUN: Ejecuta comandos en el contenedor durante el proceso de construcción de la imagen.
- COPY o ADD: Copia archivos o directorios del sistema de archivos del host al sistema de archivos del contenedor.
- WORKDIR: Establece el directorio de trabajo para las instrucciones siguientes.
- EXPOSE: Informa a Docker que el contenedor escucha en los puertos de red especificados en tiempo de ejecución.
- CMD: Especifica el comando que se ejecutará cuando se inicie un contenedor a partir de esta imagen.
- ENTRYPOINT: Configura el contenedor para que se ejecute como un ejecutable.

¿Qué es un archivo Dockerfile?

Un archivo de texto que contiene una serie de instrucciones que Docker utiliza para crear una imagen de contenedor. Esencialmente, un Dockerfile es una receta para construir una imagen Docker, especificando desde qué imagen base se debe partir, qué paquetes de software se deben instalar, cómo configurar el entorno, qué puertos exponer, y cómo ejecutar la aplicación.



```
# Usar la imagen base oficial de MySQL
```

```
FROM mysql
```

```
# Establecer las variables de entorno para la configuración de MySQL
```

```
ENV MYSQL_ROOT_PASSWORD=root_password
```

```
ENV MYSQL_DATABASE=my_database
```

```
ENV MYSQL_USER=my_user
```

```
ENV MYSQL_PASSWORD=my_password
```

```
# Copiar un archivo de inicialización opcional
```

```
# COPY ./init.sql /docker-entrypoint-initdb.d/
```

```
# Exponer el puerto 8080 para MySQL
```

```
EXPOSE 8080
```

```
# Definir el comando predeterminado para ejecutar MySQL
```

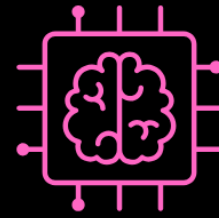
```
CMD ["mysqld"]
```

Explicación del docker-compose.yml

Nota: La imagen siguiente es un ejemplo de como funciona docker-compose.yml

¿Qué es docker-compose.yml?

docker-compose.yml es un archivo de configuración utilizado por Docker Compose para definir y ejecutar aplicaciones multi-contenedor. Docker Compose es una herramienta que permite definir y ejecutar aplicaciones Docker con múltiples contenedores mediante un archivo YAML. Este archivo YAML (docker-compose.yml) especifica los servicios que componen tu aplicación, junto con sus respectivas configuraciones, volúmenes, redes y demás dependencias.

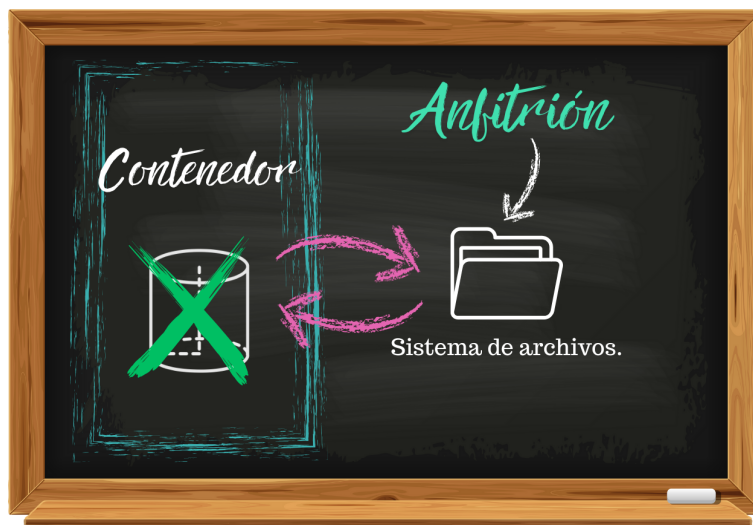


```
version: '3.8'
services:
  mysql:
    image: mysql
    container_name: mysql_container
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: my_database
      MYSQL_USER: my_user
      MYSQL_PASSWORD: my_password
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  mongo:
    image: mongo:latest
    container_name: mongo_container
    environment:
      MONGO_INITDB_ROOT_USERNAME: mongo_root
      MONGO_INITDB_ROOT_PASSWORD: mongo_password
      MONGO_INITDB_DATABASE: my_mongo_database
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

volumes:
  mysql_data:
  mongo_data:
```

- **Versión:** Define la versión del formato del archivo docker-compose.yml.
- **Servicios:** Define los servicios que queremos crear.
 - **mysql:**
 - * **image:** Usa la imagen oficial de MySQL.
 - * **container_name:** Asigna un nombre al contenedor de MySQL.
 - * **environment:** Establece las variables de entorno para configurar MySQL:
 - **MYSQL_ROOT_PASSWORD:** La contraseña para el usuario root.
 - **MYSQL_DATABASE:** El nombre de una base de datos a crear.
 - **MYSQL_USER** y **MYSQL_PASSWORD:** Un usuario y contraseña adicionales para la base de datos.
 - * **ports:** Mapea el puerto 3306 del contenedor al puerto 3306 de la máquina local.
 - * **volumes:** Define un volumen para persistir los datos de MySQL.
 - **mongo:**
 - * **image:** Usa la imagen oficial de MongoDB.
 - * **container_name:** Asigna un nombre al contenedor de MongoDB.
 - * **environment:** Establece las variables de entorno para configurar MongoDB:
 - **MONGO_INITDB_ROOT_USERNAME:** El nombre de usuario para el administrador de MongoDB.
 - **MONGO_INITDB_ROOT_PASSWORD:** La contraseña para el administrador de MongoDB.
 - **MONGO_INITDB_DATABASE:** El nombre de una base de datos a crear.
 - * **ports:** Mapea el puerto 27017 del contenedor al puerto 27017 de la máquina local.
 - * **volumes:** Define un volumen para persistir los datos de MongoDB.
- **Volúmenes:** Define los volúmenes que se usarán para almacenar los datos de MySQL y MongoDB. Podemos definir un sistema de archivos donde no se van a eliminar los datos del contenedor, cada vez que lo eliminemos.



Cómo usar el docker-compose.yml

1. Ejecuta el siguiente comando para iniciar los contenedores:

```
1 docker compose up -d
```

2. Para detener y eliminar los contenedores, ejecuta:

```
1 docker compose down
```

Algunos comandos Docker y sus Descripciones

Comando	Descripción
docker build	Construye una imagen a partir de un Dockerfile.
docker run	Ejecuta un contenedor a partir de una imagen.
docker pull	Descarga una imagen desde un repositorio (como Docker Hub).
docker push	Sube una imagen a un repositorio (como Docker Hub).
docker images	Lista todas las imágenes disponibles localmente.
docker ps	Muestra todos los contenedores en ejecución.
docker ps -a	Muestra todos los contenedores, tanto en ejecución como detenidos.
docker rm	Elimina uno o más contenedores.
docker rmi	Elimina una o más imágenes.
docker stop	Detiene un contenedor en ejecución.
docker start	Inicia un contenedor detenido.
docker restart	Reinicia un contenedor.
docker exec	Ejecuta un comando en un contenedor en ejecución.
docker logs	Muestra los logs de un contenedor.
docker inspect	Muestra la configuración detallada de un contenedor o imagen.
docker network	Gestiona redes de Docker.
docker volume	Gestiona volúmenes de Docker.
docker-compose	Ejecuta y gestiona aplicaciones multicontenedor definidas en un archivo yml.
docker login	Inicia sesión en un registro de Docker (como Docker Hub).
docker logout	Cierra la sesión de un registro de Docker.
docker tag	Etiqueta una imagen para su identificación y versión.
docker save	Guarda una imagen en un archivo tar.
docker load	Carga una imagen desde un archivo tar.
docker search	Busca imágenes en Docker Hub.
docker cp	Copia archivos, directorios entre un contenedor y el sistema de archivos local.
docker diff	Muestra los cambios en el sistema de archivos de un contenedor.
docker stats	Muestra estadísticas en tiempo real del uso de recursos por contenedores.
docker top	Muestra los procesos en ejecución dentro de un contenedor.
docker attach	Conéctate a un contenedor en ejecución.
docker pause	Pausa todos los procesos en un contenedor.
docker unpause	Reanuda todos los procesos en un contenedor pausado.
docker rename	Cambia el nombre de un contenedor.
docker update	Actualiza las configuraciones de un contenedor.
docker system df	Muestra el uso de disco de Docker.
docker system prune	Elimina todos los contenedores, redes, imágenes y volúmenes no utilizados.
docker version	Muestra información sobre la versión de Docker instalada.
docker info	Muestra información detallada sobre la instalación de Docker.
docker history	Muestra el historial de una imagen (capas).
docker kill	Mata un contenedor en ejecución.
docker wait	Espera a que un contenedor termine y muestra su código de salida.
docker export	Exporta el sistema de archivos de un contenedor como un archivo tar.
docker import	Importa el contenido desde un archivo tar para crear una nueva imagen.
docker commit	Crea una nueva imagen a partir de los cambios en un contenedor.
docker create	Crea un nuevo contenedor pero no lo inicia.
docker swarm	Gestiona un cluster de Docker en modo Swarm.
docker service	Gestiona servicios en Docker Swarm.
docker node	Gestiona nodos en un cluster de Docker Swarm.
docker config	Gestiona configuraciones en Docker Swarm.
docker secret	Gestiona secretos en Docker Swarm.
docker plugin	Gestiona plugins de Docker.
docker context	Gestiona contextos de Docker.
docker checkpoint	Gestiona checkpoints para contenedores.

Nota: Para ejecutar los comandos tenemos que tener Docker Desktop ejecutandose.